Année 2026

RAPPORT DE PROJET

LOCALISATION SPATIALE PAR TEMPS DE VOL DANS LE CAS DE DÉCHETS MARINS

ALBERT Aubin

CAMPARDOU Zoé

EL KHATIBI EL HANANA Anas

IDIAQUEZ SONG Michel

LEMAIRE Adrien

Encadrés par : Mr DAMON François

TOULOUSE INP

prepa t²

Prépa Toulouse Transitions

Remerciements

Nous remercions sincèrement notre directeur Marc Pantel pour son aide, sa bienveillance et tout le matériel qu'il nous a gracieusement prêté.

Nous adressons également nos remerciements à Morgane Kopitsch pour son aide dans l'acquisition du matériel nécessaire à la réalisation du projet, ainsi qu'à Johan Pinaud pour son soutien lors de moments cruciaux.

Nous remercions aussi toutes les intervenantes présentes lors des différents ateliers (bibliographique et réflexif) pour leur bienveillance, leur expertise et les différentes pistes de réflexions qu'elles nous ont données.

Nous tenons aussi à exprimer notre immense gratitude à notre tuteur de projet François Damon pour le suivi qu'il nous a accordé tout au long de ce projet, ainsi que pour sa pédagogie et le soutien constant qu'il nous a témoigné à chaque étape de notre travail.

Enfin, nous remercions la Prépa T^2 pour cette opportunité et de nous avoir fourni l'ensemble des ressources matérielles et infrastructurelles nécessaires à la conduite de ce projet.

Abstract

Marine pollution is a major environmental issue, with millions of tons of waste entering the seas and oceans annually. This challenge becomes more alarming when we take into consideration the difficulty of accurately locating these debris. Currently, the technological solutions at our disposal for this task are limited by a lack of development. To address this alarming issue, this study puts forth the development of an autonomous drone model that is capable of detecting and mapping marine pollution. The device relies on the Time-of-Flight (ToF) principle, utilizing a LiDAR TF-Luna infrared sensor to detect irregularities at the surface of the ocean, coupled with two HC-SR04 ultrasound sensors to enable the drone to locate itself on a two dimensional grid. For our model, data collection is managed by an Arduino microcontroller which is later on processed using Python, notably employing a Kalman filter to smooth out sensor noise and improve the reliability of the trajectory extracted from the device. This paper includes the results of the tests conducted during the research and the limitations encountered, validating the basic operating principles: the system can locate itself accurately on a 2D plane and efficiently detect the majority of tested materials, generating a pollution map in real-time. Nevertheless, the model presented limits when faced with transparent plastics and a particular sensitivity to the variations of the water surface. Overall, the prototype demonstrates the feasibility of a low-cost and low-energy solution, that is deployable at a large scale, to map zones with accumulated sea waste, with a possibility of extending the detection to not only surface-level waste.

Keywords : Infrared/ultrasound sensor, marine pollution detection, time-of-flight, spatial mapping, Python, Arduino.

Sommaire

Remerciements..... 1
Abstract..... 3
Introduction..... 5
I. Principe de fonctionnement..... 6
1. Contexte social et écologique..... 6
2. Objectifs du dispositif et choix techniques..... 6
a. Méthode du temps de vol..... 7
II. Traitement du signal LiDAR pour la détection de déchets..... 8
1. Acquisition des données..... 8
2. Mesure du niveau de référence..... 9
3. Détection d'un déchet..... 10
III. Cartographie 2D par fusion des données LiDAR et ultrasons..... 11
1. Première méthode..... 11
2. Deuxième méthode..... 12
3. Filtre de Kalman..... 13
IV. Modèle sans fil et sans Arduino (ESP32)..... 16
V. Expériences menées..... 18
1. Première expérience..... 18
2. Deuxième expérience..... 19
3. Troisième expérience..... 20
Conclusion..... 22
Bibliographie..... 23
Annexes..... 24
Relevé de décisions de la Réunion 1..... 24
Relevé de décisions de la Réunion 2..... 24
Relevé de décisions de la Réunion 3..... 24
Relevé de décisions de la Réunion 4..... 24
Code de la détection des déchets uniquement via l'Arduino IDE (cf Partie II)..... 25
Code de la cartographie des déchet via Python (cf Partie III)..... 27
Code ESP 32 (cf Partie IV)..... 43

Introduction

La pollution marine constitue aujourd'hui l'une des problématiques environnementales les plus préoccupantes de notre époque. Le Great Pacific Garbage Patch, aussi appelé "septième continent" en est un exemple alarmant. Ce "continent" concentre à lui seul près de 80 000 tonnes de plastique en suspension qui, sous l'effet des courants marins, se sont amassés en un immense nuage de déchets, s'étendant sur une superficie estimée à 1,6 million de km^2, soit trois fois la taille de la France. Ce n'est cependant qu'une manifestation de la pollution marine parmi tant d'autres : selon des estimations, entre 4,8 et 12,7 millions de tonnes de plastiques terrestres polluent les océans chaque année [1]. La dégradation de ces déchets génère des microplastiques dont la propagation dans les océans perturbe les écosystèmes marins à tous les niveaux de la chaîne alimentaire.

La localisation précise de ces amas de plastique demeure à ce jour un véritable défi, en l'absence de solution technologique réellement efficace.

C'est dans ce contexte que s'inscrit notre projet de drone capable de détecter et de localiser des déchets de manière autonome en milieu marin.

Grâce à une association de capteurs ultrasons et lumineux, et en s'appuyant sur la technologie de programmation Arduino, le drone peut détecter et cartographier les zones de pollution survolées.

Ce rapport présente dans un premier temps le contexte et les objectifs du modèle développé, avant de détailler les choix techniques et codes utilisés afin de parvenir à notre objectif de localisation de déchets.

I. Principe de fonctionnement

1. Contexte social et écologique

Chaque année, entre 4,8 et 12,7 millions de tonnes de plastiques terrestres sont déversés dans les océans. Leur traitement et leur localisation demeurent un défi majeur, compte tenu des ressources technologiques existantes. En effet, les méthodes de localisation des déchets marins sont très peu développées, étant donné les contraintes de superficie de recherche et d'imprévisibilité des trajectoires des déchets. Une solution existante est l'utilisation d'un satellite capable de détecter la couleur de l'eau et donc les amas de déchets dans l'eau [2]. Cette méthode reste cependant limitée, car il est impossible d'utiliser un satellite en présence de nuages ou la nuit. Sa résolution est aussi très faible compte tenu de la distance du satellite par rapport à l'océan. Par ailleurs, des modèles de dérive permettent d'indiquer les trajectoires 3D d'un déchet en mer en identifiant leur source, leur répartition au large et leur temps passé en mer. Cependant, ces calculs restent des modèles, et peuvent parfois être éloignés de la réalité.

Ainsi, comment notre modèle se distingue-t-il des solutions déjà existantes ? Comment dépasser les limites technologiques ?

2. Objectifs du dispositif et choix techniques

Pour remédier au problème majeur que pose la localisation des déchets, nous avons souhaité créer un prototype de drone capable de détecter les déchets en mer.

Pour ce faire, nous avons fait le choix d'utiliser les interfaces programmables Arduino, qui permettent, par l'intermédiaire d'un microcontrôleur, de réaliser différents projets grâce à une plateforme électronique et un circuit programmable. Pour nos capteurs, nous en avons choisi un pour la détection de déchets (LiDAR) et deux pour la localisation dans l'espace (ultrasons HC-SR04). Le choix d'utiliser un capteur de type LiDAR repose principalement sur le fait qu'il utilise le temps de vol. Cette méthode est non seulement le sujet de notre projet, mais aussi très rapide dans son exécution. De plus, un capteur

infrarouge nous donne la possibilité d'utiliser notre dispositif la nuit. Nous avons choisi le modèle TF-Luna par Benewake parce que c'était le seul capteur qui respecte notre budget. Le choix des capteurs HC-SR04 a été assez opportuniste. Nous voulions utiliser un autre type de capteur pour éviter les interférences et, les capteurs ultrasons étant déjà disponibles, quelques essais ont suffi à nous montrer que la précision et la praticité de ces capteurs (par ailleurs très bien documentés) dépassaient nos attentes.

a. Méthode du temps de vol

La méthode du temps de vol (*Time of Flight ou aussi ToF en anglais*) est une technique de télémétrie optique active, rapide et adaptable à tout type de surface, ce qui explique son utilisation dans des domaines variés comme la robotique, les véhicules autonomes ou encore les dispositifs de reconnaissance spatiale. Ce principe consiste à émettre une onde lumineuse, généralement dans le domaine infrarouge, qui se réfléchit sur la surface de l'obstacle à détecter et revient dans le capteur, et à mesurer le temps écoulé avant la réception du signal en retour [3]. Cette mesure de temps permet de déduire simplement la distance entre l'émetteur et l'obstacle grâce à la relation : $distance = \frac{célérité \times durée}{2}$. [4]
Cette méthode permet de mesurer facilement une distance à un obstacle, de manière non intrusive et sans aucune perturbation de l'environnement autour, combinée avec une rapidité d'exécution compatible avec notre système embarqué.

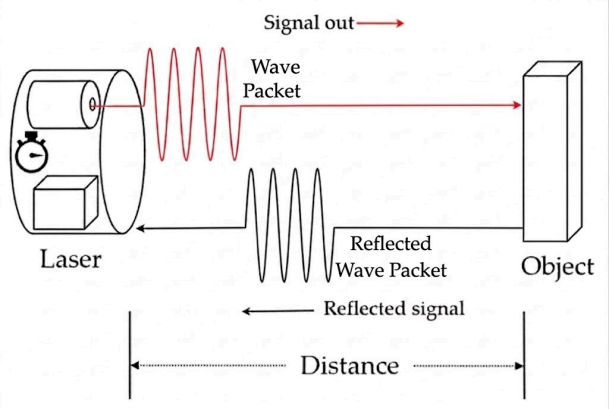


Figure 1 : Schéma de fonctionnement d'un capteur ToF [5]

II. Traitement du signal LiDAR pour la détection de déchets

Dans cette partie le langage utilisé est le C++, afin qu'il puisse être interprété par le logiciel Arduino IDE, logiciel nécessaire au bon fonctionnement de nos composants.

Tout d'abord, rappelons que pour détecter un déchet nous étudions la différence entre la distance mesurée par le capteur et un niveau de référence défini préalablement.

Il faut aussi savoir que, une fois le programme téléversé sur la carte arduino, le code suivant `void setup()` n'est exécuté qu'une seule fois alors que le code suivant `void loop()` s'exécute en boucle tant que la carte reste branchée.

1. Acquisition des données

Afin de prélever et d'interpréter les données mesurées par le TF-Luna, nous avons besoin d'utiliser cette partie du code qui va être expliquée juste après :

```
void loop() {
  if (tfLuna.available() >= 9) {
    if (tfLuna.read() == 0x59) {
      if (tfLuna.peek() == 0x59) {
        tfLuna.read(); // Consomme le deuxième 0x59

        uint8_t low = tfLuna.read();
        uint8_t high = tfLuna.read();
        int dist = low + (high << 8);
        for(int i=0; i<5; i++) tfLuna.read(); // Vide la trame
}
```

Le TF-LUNA communique par le biais d'une liaison série, gérée ici par la bibliothèque *SoftwareSerial* [6], sur les pins 2 et 3.

Chaque mesure complète est contenue dans 9 octets successifs. Le principal défi lors de la programmation est d'aligner la lecture sur le début de cette suite d'octets. L'appareil emploie un code particulier pour indiquer le début d'un message : deux octets de « header » (en-tête) contenant la valeur hexadécimale 0x59.

Dans notre code, nous employons la méthode “peek()” pour contrôler la présence du second indicateur sans le lire trop tôt, assurant de ce fait que nous accédons aux informations à l'endroit approprié. La distance est par la suite reconstituée à partir des deux octets suivants.

Puisqu'un octet a forcément une valeur allant de 0 à 255, le capteur divise la mesure en un octet de poids faible (“low”) et un octet de poids fort (“high”). Nous utilisons donc un décalage de bits (“high << 8”), ce qui revient à multiplier la valeur par 2^8 (donc 256). L'opération finale se résume à : $dist = low + (high * 256)$.

Nous avons établi une fourchette de validité allant de 10 à 5000 centimètres. Ceci permet de supprimer les fautes de lecture occasionnelles qui pourraient se produire si le détecteur est trop près d'un obstacle ou si le signal est temporairement interrompu. Dès que la validité est attestée, la première mesure stable identifiée est assignée à la variable “distanceCalibree”, établissant ainsi un point de référence pour l'ensemble de la session de détection. Pour assurer la réactivité du système, nous avons incorporé une procédure de nettoyage du tampon de réception. Si la mémoire de la liaison série dépasse 15 octets, l'excédent est purgé.

Cette mesure est essentielle parce que le TF-LUNA transmet des informations à une vitesse supérieure à celle de notre cycle “void loop”. Sans cette opération de nettoyage, le drone traiterait des informations obsolètes avec un retard temporel, ce qui compromettrait la précision de l'identification des déchets pendant le vol du drone.

2. Mesure du niveau de référence

Nous calibrons la distance dans cette boucle donc il faut lui attribuer une valeur uniquement lorsque aucune valeur ne lui a encore été attribuée (distanceCalibree = 0). Si nous ne faisions pas cela, à chaque répétition de la boucle une nouvelle distance de référence va être mesurée, ce que nous souhaitons éviter.

```
int distanceCalibree = 0;
void loop() {
if (distanceCalibree == 0) {
            distanceCalibree = dist;
            Serial.print("calibration faite sur : ");
            Serial.print(distanceCalibree);
}
```

Ces lignes de code servent donc à mesurer le niveau de référence du drone par rapport à l'eau.

3. Détection d'un déchet

Après avoir mesuré une distance de référence, nous déplaçons le capteur à une altitude constante en vérifiant. Simultanément, l'algorithme vérifie si l'écart entre la distance mesurée et celle de référence ne dépasse pas un certain seuil.

Ce seuil est déterminé en variable globale.

```
const int seuil = 2; // On met un seuil en cm
void loop() {
          int difference = distanceCalibree - dist;
          if (difference >= seuil) {
           Serial.println("!!!!! DECHET DETECTE !!!!!");
     }
      }
```

Nous avons choisi de mettre un seuil afin d'augmenter la robustesse de notre système de détection. En effet, on prend ainsi en compte le fait que le capteur puisse ne pas renvoyer exactement les mêmes valeurs de distance même à altitude constante.

Dans le cas où cette différence est supérieure au seuil, cela veut dire qu'un obstacle se trouve en dessous (que l'on suppose ici être forcément un déchet).

III. Cartographie 2D par fusion des données LiDAR et ultrasons

Pour cartographier les déchets, nous avons eu deux contraintes principales. Premièrement, il nous fallait localiser le drone dans l'espace. Nous avons choisi d'utiliser deux capteurs ultrason HC-SR04 placés à 90° qui relèveraient deux distances avec deux points fixe, afin de simuler deux bateaux au large qui renverraient les ondes émises (cf : Première expérience).

Deuxièmement, la console de l'Arduino IDE ne permet pas de faire des interfaces graphiques. Nous avons choisi d'utiliser le langage Python, qui dispose de toutes les librairies nécessaires.

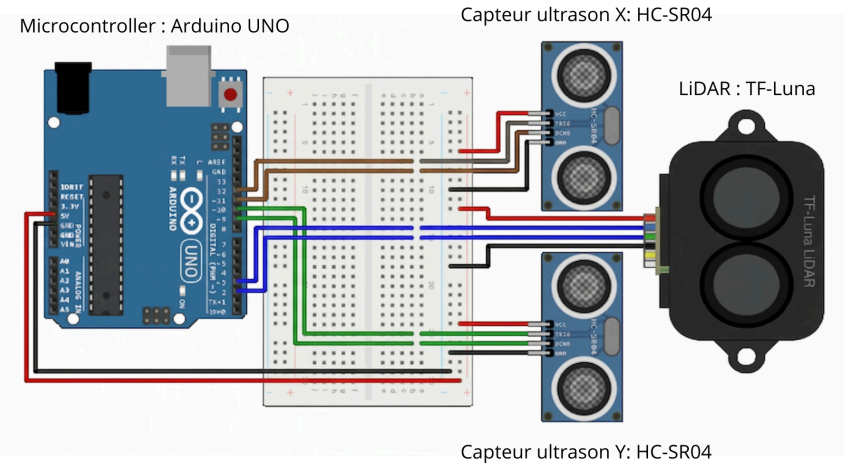


Figure 2 : Schéma de branchement de la carte Arduino UNO

1. Première méthode

Nous avons essayé plusieurs méthodes pour transférer les données acquises par les capteurs à Python. Notre premier essai a été d'utiliser la bibliothèque *telemetrix* [7], qui permet de commander directement la carte depuis Python. Les capteurs ultrasons étant des capteurs très communs, nous n'avons pas eu de problème pour les utiliser dans Python. Un problème majeur est apparu quand nous avons essayé de lier le LiDAR. Le LiDAR a une vitesse de fonctionnement très élevée et, là où la carte peut gérer cet afflux de données, c'est beaucoup plus dur pour Python avec la bibliothèque *telemetrix*, et à chacun

de nos essais, au bout de quelques secondes, Python se sécurise en bloquant le port d'entrée d'information.

2. Deuxième méthode

Nous avons donc fait plus de recherches, et avons découvert une méthode bien plus simple. Au lieu de faire faire l'acquisition et le traitement initial des données à Python, nous avons laissé la carte le faire, puis nous avons renvoyé les différentes données de distance à Python. Même si une acquisition des données des capteurs ultrasons en parallèle de celle du LiDAR serait techniquement réalisable, nous avons choisi, pour éviter des perturbations et pour simplifier le système, de faire une acquisition en "série". L'acquisition des données du LiDAR se fait de la même manière que précédemment, dans la partie II, puis une fois les valeurs aberrantes omises, nous passons à l'acquisition des données ultrasons. Le code d'acquisition d'un capteur ultrason est le suivant :

```
const int trigPin = 10; // Pin arduino gérant l'émission du signal
const int echoPin = 9; // Pin arduino gérant la réception du signal

digitalWrite(trigPi1, LOW);  // Stabilisation du signal
delayMicroseconds(5);
digitalWrite(trigPin, HIGH); // Déclenchement de l'impulsion
delayMicroseconds(10);         // Durée d'impulsion requise pour le capteur
digitalWrite(trigPin, LOW);  // Arrêt de l'impulsion

duree = pulseIn(echoPin, HIGH, 30000); // Mesure le temps de retour (timeout de
30ms pour éviter de bloquer la carte)
distance = duree1*0.034/2; // d = v * t * 1/2, v la vitesse du son  (0.034 m/ms)
et 1/2 pour prendre en compte l'aller-retour
```

Pour envoyer les données à Python nous passons par la commande `Serial.println(XXX)` Cette commande envoie "XXX" sur un "port série" (une interface de communication), qui peut être facilement lu en python à l'aide de la bibliothèque *pyserial*[8] et des ligne de commandes suivantes:

```
import serial
arduino = serial.Serial(port_arduino, vitesse, timeout=1) #Ouverture de l'interface série sur le port_arduino, à une certaine vitesse de transmission
donnee = arduino.readline().decode('utf-8').rstrip() #Lecture de la ligne reçue
```

Nous récupérons donc `donnee`, composé des trois coordonnées x, y et z que nous pouvons utiliser. Après la première expérience, nous nous sommes aperçus d'une grande variabilité sur les acquisitions des capteurs ultrason, qui rendait peu précise la cartographie. Pour améliorer nos résultats, nous avons exploré plusieurs filtres possibles. Dans un premier temps, nous avons essayé d'implémenter une moyenne glissante sur les *n* dernières valeurs, pour lisser les résultats. Pour des valeurs de *n* supérieur à 10, la variabilité baisse beaucoup, mais nous avons observé une forte latence sur les résultats, et nous n'avons pas trouvé de valeur de *n* qui nous satisfasse à la fois en termes de lissage et de réactivité. Nous avons donc cherché d'autres façons de filtrer nos données, et nous avons implémenté le filtre dit *de Kalman* (implémentation trouvable en annexe).

3. Filtre de Kalman

Avant de directement utiliser un filtre de Kalman, nous avons choisi d'implémenter un filtre intermédiaire pour faire un tri anticipé des valeurs mesurées. Le filtre intermédiaire garde en mémoire les N dernières valeurs mesurées et en extrait la médiane. Ce filtre a été mis en place pour exclure les valeurs aberrantes (ex: 1000 cm). On préfère cette méthode a une moyenne classique parce que cette dernière sera faussée par une valeur extrême.

Le filtre Kalman[9] va lisser le bruit (tremblements ou incertitudes des capteurs). Il fonctionne grâce à deux paramètres principaux : l'erreur de mesure et l'imprévisibilité de l'environnement notées R et Q respectivement. Ces paramètres sont fixes que l'on impose avant l'exécution du code.

Le choix du R est relié à la confiance que l'on a envers nos capteurs. Plus R est grand, moins le filtre va faire confiance dans les mesures qu'il reçoit, ce qui va lisser fortement la trajectoire mais aussi augmenter la latence.

Notre Q va déterminer comment votre filtre va réagir face aux mouvements brusques du "drone". Plus Q est petit, le filtre va ignorer les changements abrupts de position et diminuer les tremblements.

Nous avons choisi les valeurs de R et Q empiriquement, en trouvant un compromis entre latence des résultats, lissage et précision des valeurs

Le filtre Kalman s'effectue avec deux étapes principales, la prédiction et la correction. Ces étapes sont effectuées de manière récursive. Au coeur de notre code, on trouve ces quatre lignes :

```
prediction = self.incertitude + self.q  # 1. Étape de Prédiction
k = prediction / (prediction + self.r)      # 2. Étape de Correction
self.position = self.position+k*(mesure-self.position)# Mise à jour de la position
self.incertitude = (1 - k) * prediction # Mise à jour de l'incertitude
```

$$P_{pred} = P_{k-1} + Q$$

P_{k-1} : “self.incertitude” (niveau d’incertitude du tour précédent).
Q : “self.q” (cf. précédemment).
P_{pred} : “prediction” (le nouveau niveau d’incertitude).

L'étape de prédiction a lieu avant la lecture des mesures fournies par les capteurs, notre algorithme a pour but d'essayer de "deviner" la localisation du drone. Notre incertitude sur la position du drone va augmenter en anticipant les possibles variations involontaires de sa trajectoire.

$$K = \frac{P_{pred}}{P_{pred} + R}$$

R : "self.r" (cf. précédemment).

K : le Gain de Kalman.

P_{pred} : "prediction" (le nouveau niveau d'incertitude).

Pendant l'étape de correction, on ne calcule pas une position mais un "pourcentage de confiance", qui correspond au Gain de Kalman (K). Ici on compare notre incertitude (P_{pred}) à la "somme totale de toutes les incertitudes" ($P_{pred} + R$). Le gain de Kalman nous permettra de déterminer l'importance qu'il faudra accorder aux valeurs erratiques du capteur. Plus le gain se rapproche de 0, plus le filtre va lisser davantage la trajectoire en ignorant les mesures aberrantes.

$$X_k = X_{pred} + K \times (Z - X_{pred})$$

X_k : "self.position" (la nouvelle position finale calculée).

X_{pred} : ancienne "self.position" (la position prédite avant la lecture du capteur).

Z : "mesure" (la distance brute renvoyée par le capteur).

K : le Gain de Kalman calculé à l'étape précédente.

La mise à jour de la position nous permet de calculer la coordonnée finale. Pour notre modèle nous utilisons un filtre de Kalman simplifié, on fait la supposition que X_{pred} est égale à la position lors de l'itération précédente X_{k-1}. Le terme $(Z - X_{pred})$ représente l'écart entre la mesure réalisée par le capteur et l'ancienne position du dispositif. La multiplication par le Gain de Kalman indique qu'on ne parcourra qu'un pourcentage du chemin pour corriger cette "erreur".

$$P_k = (1 - K) \times P_{pred}$$

Enfin, la mise à jour de l'incertitude est là pour calibrer le système en réduisant l'incertitude.

Une fois les données acquises et filtrées, nous avons développé différents types d'interface graphique pour cartographier les données, à l'aide de la bibliothèque *tkinter* [10]. Nous ne développerons pas leurs implémentations dans ce rapport, le sujet s'éloignant significativement de la problématique initiale. Les codes restent tout de même commentés en annexe.

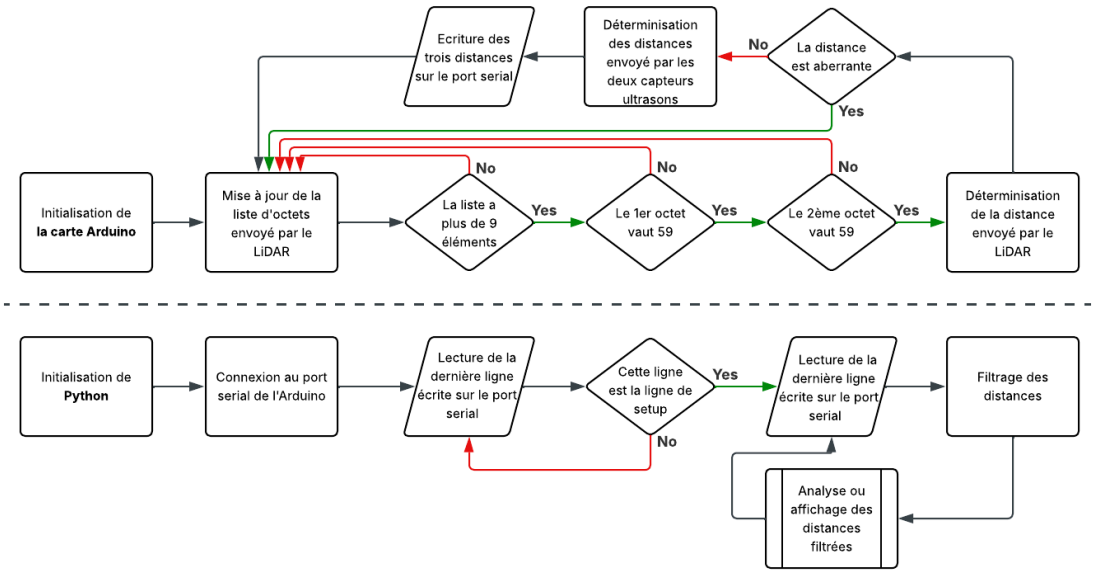


Figure 3 : Schéma fonctionnel du programme d'acquisition sur l'Arduino et du programme de traitement sur Python

IV. Modèle sans fil et sans Arduino (ESP32)

Comme dit précédemment dans nos objectifs du dispositif, la première idée de modèle qui nous est venue à l'esprit est celle d'un drone contrôlable, dédié à la détection des déchets au-dessus de la mer. Pour cela, l'élément le plus important à considérer est la communication sans fil entre le drone, qui porte les capteurs, et l'ordinateur, qui traite les données fournies par ces derniers. Dans le cas d'une carte Arduino, la liaison filaire entre

celle-ci et l'ordinateur est essentielle au bon fonctionnement des capteurs. Il est donc impossible de la rendre sans fil, d'où l'importance de l'ESP32. L'ESP32 est une série de microcontrôleurs d'*Espressif Systems* intégrant la gestion du Wi-Fi et du Bluetooth.

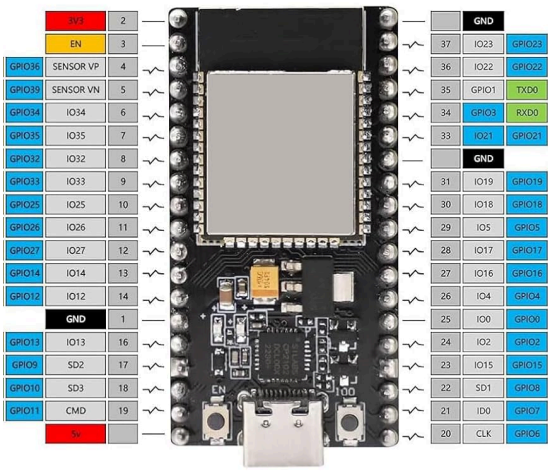
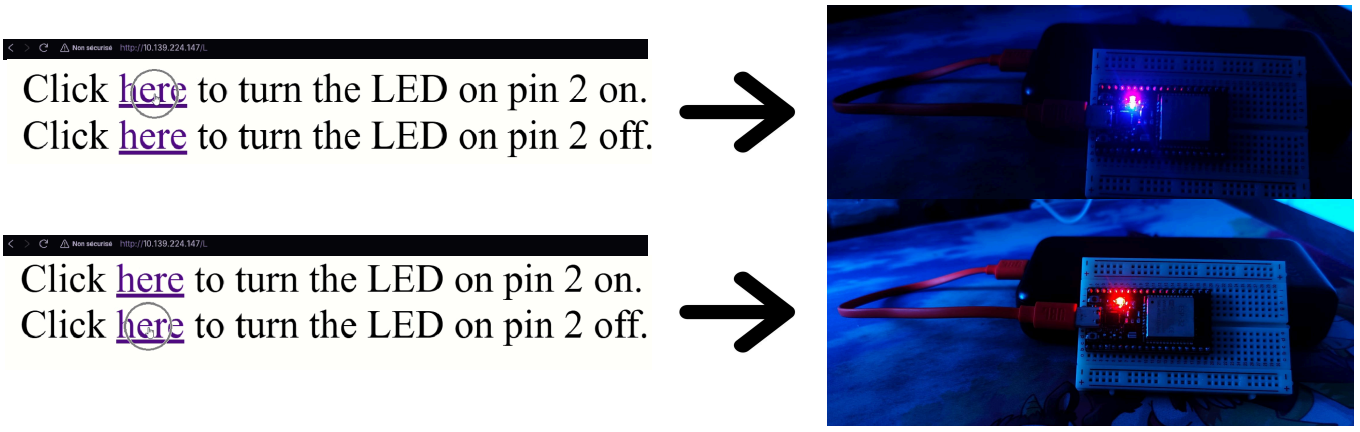


Figure 4 : Schéma de brochage de la carte de développement ESP 32 [11]

Dans l'idée, l'utilisation de cette carte avait pour objectif de remplacer l'Arduino Nano afin d'obtenir l'aspect sans fil dont il est question. En effet, une carte ESP32 peut faire tout ce que fait une carte Arduino ordinaire, voire mieux. Seulement, elle est un peu plus compliquée à manipuler que cette dernière.

La première étape pour tester l'ESP32 est d'allumer une de ses LED intégrées. Un peu comme le test "`print("Hello World")`" sur Python, cette étape permet de vérifier le bon fonctionnement des composants de la carte et donc qu'elle est en bon état. Dans notre cas, et avec un programme intégré à la carte, nous avons réussi à nous connecter via WIFI à l'ESP32 et, grâce à son adresse IP, à avoir une interface HTML qui nous permet d'éteindre et d'allumer la LED à distance.



(La LED rouge est toujours allumée, la LED à allumer est la bleue.)

Figure 5 : Schéma pratique de la vérification de l'allumage d'une LED en WIFI [12]

Après cette étape concluante, il restait seulement à relier l'ESP32 avec l'ordinateur, afin de commander la carte directement depuis Python. Nous avons donc opté pour l'utilisation de la bibliothèque *telemetrix_esp32*[13]. Cependant, de la même façon que pour le TF-Luna, beaucoup de problèmes, qui cette fois-ci étaient des problèmes de compatibilité, ont eu lieu, et, malgré les différents essais pour arranger les choses (comme changer de bibliothèque, passer d'une connectique sans fil WIFI à une connectique sans fil Bluetooth, changer le code qui met en place le point d'accès WIFI, etc), nous n'avons trouvé aucune solution. Ainsi, par manque de temps et de progression, nous avons été contraints d'abandonner l'idée d'utiliser l'ESP32 et avons donc décidé de conserver l'utilisation de notre Arduino Nano en filaire pour nos futures expériences.

V. Expériences menées

1. Première expérience

Nous avons mené une expérience de localisation en 2D pour voir si notre algorithme utilisant les capteurs à ultrasons était fonctionnel. Pour ce faire, nous avons positionné les deux capteurs à 90° pour simuler leur configuration sur le drone.

Les résultats obtenus montrent que la localisation dans un plan en 2D est possible et que l'interface graphique développée permettait de bien localiser le drone en temps réel sur une grille avec une précision satisfaisante.

Il a par ailleurs été observé que la localisation du drone reste stable pour des rotations allant jusqu'à 30°, au-delà desquelles le signal est perdu. Enfin, quand le capteur reste statique, un certain décalage dynamique apparaît, dont la correction nécessitera l'utilisation d'un filtre de Kalman dans l'algorithme.



Figure 6 : Expérience de localisation en 2D avec les capteurs ultrasons

2. Deuxième expérience

La deuxième expérience visait à évaluer les capacités de détection de différents déchets par le capteur LiDAR. Les résultats obtenus montrent une détection satisfaisante pour la majorité des matériaux testés. Cependant, les plastiques transparents (tels que ceux utilisés en emballages alimentaires) ont été difficiles à détecter. Par ailleurs, des perturbations ont été observées quand l'eau est agitée. Ces perturbations montrent la nécessité d'un traitement informatique de l'agitation de l'eau afin de pouvoir se rapprocher des conditions réelles de fonctionnement du drone. Enfin, les performances du capteur en l'absence de lumière se sont révélées semblables aux précédentes, ce qui constitue un atout pour un usage de nuit.

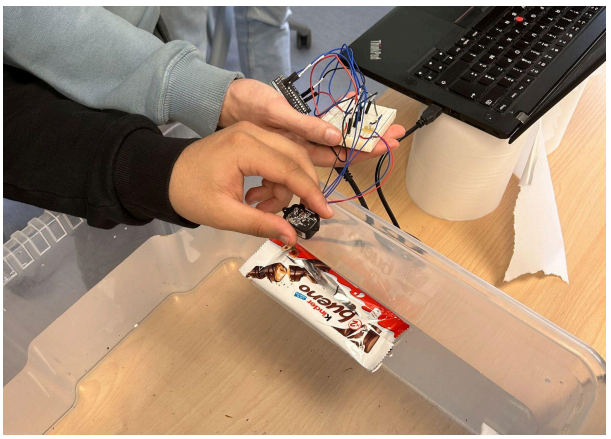
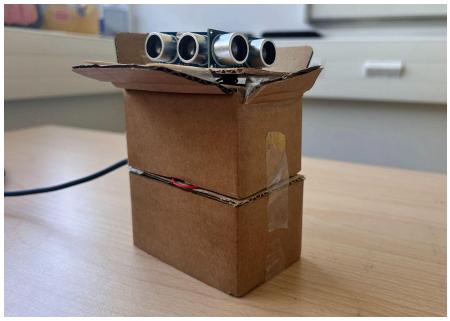
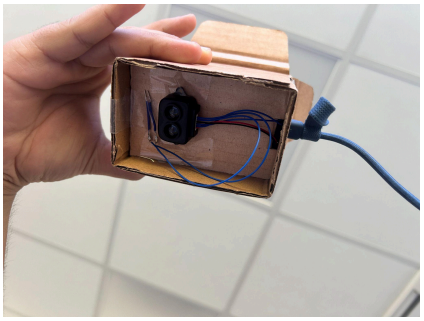


Figure 7 : Expérience de détection de déchet grâce au capteur LiDAR

3. Troisième expérience





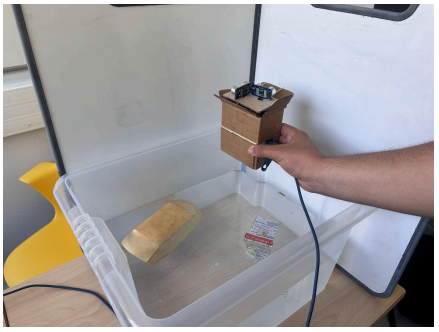


Figure 8 : (a) Vue de profil du modèle de drone avec les deux capteurs ultrasons visibles en haut.

(b) Vue de dessous du modèle avec le capteur LiDAR visible.

(c) Mise en situation de détection des déchets dans de l'eau

Lors de cette expérience finale, nous avons pu tester notre prototype en action, avec les deux types de capteurs combinés dans un modèle représentant le drone. Cette expérience a été très concluante, montrant l'efficacité de notre système de localisation des déchets. Le modèle est capable de se situer sur un plan en 2D, et grâce à une grille quadrillée, de situer des déchets ainsi que la superficie recouverte par les déchets.

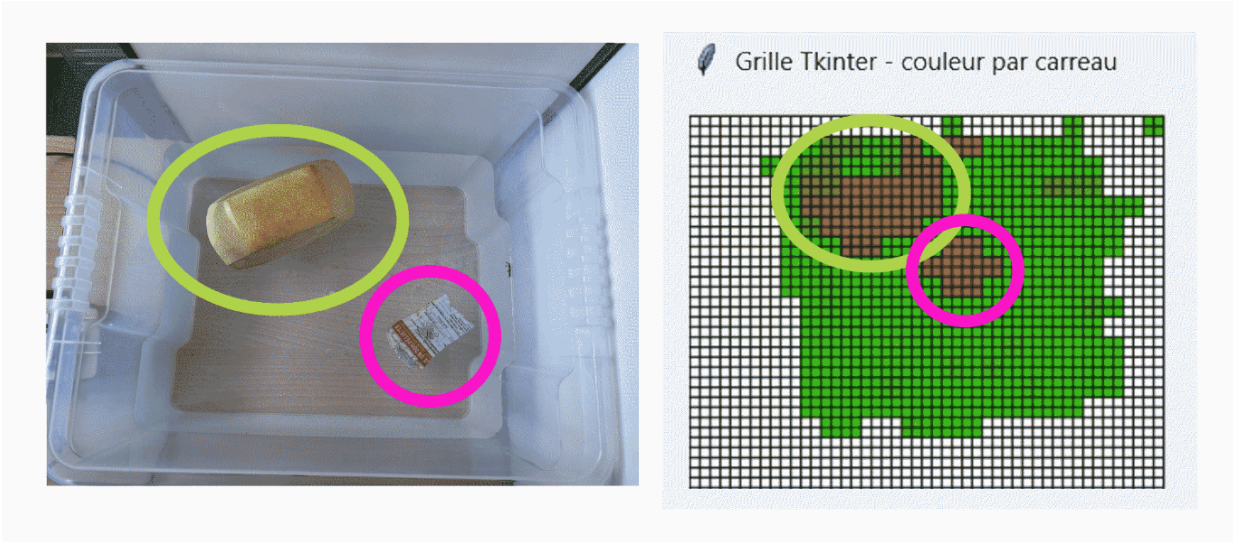


Figure 9 : Association entre la réalité et l'interface graphique de cartographie des déchets. Les deux objets paraissent plus proches sur l'interface car nous avons choisi une précision de 2 centimètres

Nous avons développé une interface graphique à l'aide de la bibliothèque Python tkinter, qui cartographie l'espace sous forme de grille 2D. Chaque case est d'une couleur distincte selon la détection ou non d'un déchet par les capteurs du drone (du vert en l'absence de déchet au rouge vif si un déchet est bien détecté). Cette représentation permet une cartographie efficace et intuitive des déchets présents dans la mer.

Analyse réflexive

La répartition des tâches s'est globalement bien passée au sein du groupe. Chaque membre avait des compétences qui étaient complémentaires aux autres, que ce soit d'ordre technique ou rédactionnel, ce qui nous a permis de collaborer efficacement et de manière organisée.

La principale difficulté rencontrée durant tout le projet a été l'organisation : nous avons dû jongler entre les concours, les examens de fin de semestre et un léger manque de motivation du début de projet. Avec le recul, certaines phases du projet auraient pu être abordées plus tôt, et une meilleure répartition des tâches dès le début nous aurait fait

travailler plus efficacement et nous aurait permis de mieux approfondir certains aspects que nous n'avons que vaguement évoqués. Ce projet nous a appris que la réussite d'un travail de groupe ne repose pas seulement sur la maîtrise du sujet mais aussi sur la communication, l'entente, l'investissement du groupe et la bonne volonté de chacun pour donner du sien afin d'avancer. Cette prise de conscience collective a constitué le véritable moteur de notre progression au fil du projet.

Conclusion

Les expériences menées au cours de ce projet ont permis de valider les principes fondamentaux de notre système. D'une part, la localisation en deux dimensions du drone s'est révélée fonctionnelle, et d'autre part le capteur LiDAR a démontré des capacités de détection satisfaisantes pour la majorité des déchets ciblés. Cependant, plusieurs limites ont été mises en évidence par nos expériences au cours du projet. En effet, les plastiques transparents sont encore mal détectés par le capteur LiDAR et le système est très sensible aux perturbations à la surface de l'eau. Ces limites constituent des pistes d'amélioration pour des développements plus poussés du projet.

A plus long terme, on pourrait envisager d'étendre la localisation à trois dimensions afin d'obtenir une cartographie plus précise des zones de pollution, et de tester ce prototype en conditions réelles en milieu marin pour anticiper des difficultés supplémentaires éventuelles.

Replacé dans le contexte environnemental et sociétal qui a motivé ce projet, ce prototype démontre la faisabilité d'une solution efficiente, peu énergivore, et déployable facilement à grande échelle pour la cartographie des zones d'accumulation de déchets. Ce projet, bien qu'expérimental, s'inscrit dans une démarche de lutte contre la pollution marine, phénomène de grande ampleur dont les dégâts sont chaque année plus importants dans tous les domaines.

Bibliographie

1. Jambeck, J. R.; Geyer, R.; Wilcox, C.; Siegler, T. R.; Perryman, M.; Andrady, A.; Narayan, R. and Law, K. L. Plastic waste inputs from land into the ocean. *Science* 347(6223): 768-771, https://doi.org/10.1126/science.1260352.
2. Large-scale detection of marine debris in coastal areas with Sentinel-2 Rußwurm, Marc et al. iScience, Volume 26, Issue 12, 108402
3. Grzegorzek, M.; Theobalt, C.; Koch, R. and Kolb, A. *Time-of-Flight and Depth Imaging. Sensors, Algorithms, and Applications*. In Lecture Notes in Computer Science; Springer: Berlin, Heidelberg, 2013.
4. Jie, M.; Shenglong, Z.; Lei, Q.; Yuzhu, G.; Yifan, W.; Ming, Z.; Rui, B.; Miao, S. and Patrick, Y. C. A review of ToF-based LiDAR. *Journal of Semiconductors* 45(10): 101201, https://doi.org/10.1088/1674-4926/24040015.
5. Zheng, Ling & li, Bijun & Yang, Bo & Song, Huashan & Lu, Zhi. (2019). Lane-Level Road Network Generation Techniques for Lane-Level Maps of Autonomous Vehicles: A Survey. Sustainability. 11. 4511. 10.3390/su11164511.
6. SoftwareSerial Library : Arduino. Emulated serial ports. Retrieved from https://www.arduino.cc/en/Reference/SoftwareSerial (accessed April 6, 2026).
7. Telemetrix : Shifman, A. Remote Arduino control. Retrieved from https://github.com/MrYsLab/telemetrix (accessed April 6, 2026).
8. Serial (PySerial) : Liechti, C. Device communication interface. Retrieved from https://pyserial.readthedocs.io/ (accessed April 6, 2026).
9. Welch, G., & Bishop, G. (2006). *An Introduction to the Kalman Filter*. University of North Carolina at Chapel Hill, Department of Computer Science, https://www.cs.unc.edu/~welch/media/pdf/kalman_intro.pdf
10. Python Software Foundation. Graphical User Interfaces with Tk. Retrieved from https://docs.python.org/3/library/tkinter.html (accessed April 6, 2026).
11. Detailed pinout diagram of the ESP32 Development Board, illustrating the function of each of the 38 pins for various applications. Retrieved from https://manuals.plus/asin/B0DG8JSS7V (accessed April 9, 2026).
12. GitHub, dépôt Espressif Systems. SimpleWiFiServer.ino (code Arduino IDE pour ESP32). Retrieved from https://github.com/espressif/arduino-esp32/blob/master/libraries/WiFi/examples/SimpleWiFiServer/SimpleWiFiServer.ino (accessed April 8, 2026). (voir le code annexe dans Code ESP 32)
13. Telemetrix for The ESP32 : Shifman, A. Remote sensor control. Retrieved from https://github.com/MrYsLab/telemetrix-esp32 (accessed April 6, 2026).

Annexes

Relevé de décisions de la Réunion 1

- Faire un cahier des charges
- Améliorer la recherche bibliographique
- S'améliorer pour les prochaines réunions (ordre du jour…)
- Choisir les pièces à commander avant le 16 février

Relevé de décisions de la Réunion 2

- Avancer sur les manipulations
- Utiliser plusieurs capteurs
- Se répartir les manipulations informatiques : 1 groupe LiDAR et 1 groupe ultrason
- Mettre un responsable codage
- Programmer une séance en salle de TP pour utiliser l'oscilloscope

Relevé de décisions de la Réunion 3

- Commencer à rédiger le rapport, en incluant des résumés d'expériences
- Réfléchir au plan et à la problématique
- Tester les limites des capteurs
- Faire la base de données sur les différents matériaux détectables par le LiDAR
- Travailler sur la transformation du code Arduino vers Python

Relevé de décisions de la Réunion 4

- Prévoir un entraînement à la soutenance orale avec le tuteur
- Retravailler les citations bibliographiques
- Retravailler certains points du rapport

Code de la détection des déchets uniquement via l'Arduino IDE (cf Partie II)

En C++:

```
#include <SoftwareSerial.h>


SoftwareSerial tfLuna(2, 3);


int distanceCalibree = 0;
const int seuil = 2; // On met un seuil en cm


void setup() {
  Serial.begin(115200);
  tfLuna.begin(115200);
  delay(3000); // On laisse 3 secondes pour bien stabiliser
  Serial.println("--- Setup OK ---");
}


void loop() {
  delay(100);
  if (tfLuna.available() >= 9) {
    if (tfLuna.read() == 0x59) {
      if (tfLuna.peek() == 0x59) {
        tfLuna.read(); // Consomme le deuxième 0x59

        uint8_t low = tfLuna.read();
        uint8_t high = tfLuna.read();
        int dist = low + (high << 8);
        for(int i=0; i<5; i++) tfLuna.read(); // Vide la trame


        // Filtre les valeurs aberrantes
        if (dist > 10 && dist < 5000) {

          // Si c'est la première valeur valide, on calibre
          if (distanceCalibree == 0) {
            distanceCalibree = dist;
            Serial.print("calibration faite sur : ");
```

```
            Serial.print(distanceCalibree);
            Serial.println(" unités");
          }


          // CALCUL DE LA DIFFÉRENCE
          int difference = distanceCalibree - dist;


          Serial.print("Dist: ");
          Serial.print(dist);
          Serial.print(" | Ref: ");
          Serial.print(distanceCalibree);
          Serial.print(" | Diff: ");
          Serial.println(difference);


          if (difference >= seuil) {
            Serial.println("!!!!! DÉCHET DÉTECTÉ !!!!!");
          }
        }
      }
    }
  }


  // Nettoyage rapide du tampon
  if (tfLuna.available() > 15) {
    while(tfLuna.available()) tfLuna.read();
  }
}
```

Code de la cartographie des déchet via Python (cf Partie III)

Le projet est trouvable en intégralité sur GitHub : github.com/Anorak33/Projet-2A-dechet

En C++ :

arduino/arduino.ino :

```
#include <SoftwareSerial.h>

const int trigPinX = 8; // Trigger (émission du signal)
const int echoPinX = 7; // Echo (réception du signal)
const int trigPinY = 10; // Trigger (emission)
const int echoPinY = 9; // Echo (réception)

const int offsetX = 1; // Décalage en cm entre le capteur ultrason X et le centre du LiDAR
const int offsetY = 1; // Décalage en cm entre le capteur ultrason Y et le centre du LiDAR

SoftwareSerial tfLuna(2, 3);


long dureeX; // durée de l'echo
long dureeY;
int distanceX;  // distance calculée en cm
int distanceY;

void setup() {
  Serial.begin(115200);
  tfLuna.begin(115200);
  delay(1000); // On laisse 3 secondes pour bien stabiliser

  pinMode(trigPinX, OUTPUT); // Configuration du port du TriggerX comme une SORTIE
  pinMode(echoPinX, INPUT); // Configuration du port de l'EchoX comme une ENTREE
  pinMode(trigPinY, OUTPUT); // Configuration du port du TriggerY comme une SORTIE
  pinMode(echoPinY, INPUT); // Configuration du port de l'EchoY comme une ENTRÉE

  Serial.println("--- Setup OK ---");
}

void loop() {
  if (tfLuna.available() >= 9) {
```

```
    if (tfLuna.read() == 0x59) {
      if (tfLuna.peek() == 0x59) {
        tfLuna.read(); // Consomme le deuxième 0x59
        uint8_t low = tfLuna.read();
        uint8_t high = tfLuna.read();
        int distanceZ = low + (high << 8);
        for(int i=0; i<5; i++) tfLuna.read(); // Vide la trame

        // Filtre les valeurs aberrantes
        if (distanceZ > 0 && distanceZ < 255) {
          digitalWrite(trigPinX, LOW);  // Stabilisation du signal
          delayMicroseconds(5);
          digitalWrite(trigPinX, HIGH); // Déclenchement de l'impulsion
          delayMicroseconds(10);        // Durée d'impulsion requise pour le capteur
          digitalWrite(trigPinX, LOW);  // Arrêt de l'impulsion

          // Mesure le temps de retour (timeout de 50ms pour éviter de bloquer la carte)
          dureeX = pulseIn(echoPinX, HIGH, 50000);
          distanceX = dureeX*0.034/2 + offsetX; // d = v * t * 1/2, v la vitesse du son
(0.034 m/ms) et 1/2 pour prendre en compte l'aller-retour

          digitalWrite(trigPinY, LOW);
          delayMicroseconds(5);
          digitalWrite(trigPinY, HIGH);
          delayMicroseconds(10);
          digitalWrite(trigPinY, LOW);

          dureeY = pulseIn(echoPinY, HIGH,50000);
          distanceY = dureeY*0.034/2 + offsetY;


          Serial.print(distanceZ);
          Serial.print(",");
          Serial.print(distanceX);
          Serial.print(",");
          Serial.println(distanceY);
        }
      }
    }
  }
```

```
  // Nettoyage rapide du tampon
  if (tfLuna.available() > 15) {
    while(tfLuna.available()) tfLuna.read();
  }
}
```

En Python :

main.py : (programme principal)

```
"""Programme principal du projet de détection des déchets."""

from types import ModuleType, FunctionType
from importlib import import_module

from constante import *
from lecture_serial import *
from kalman import filtrage_kalman



def main(affichage:FunctionType)->None:
    """Fonction principale du programme. Elle gère la connexion avec l'Arduino, la lecture des données, le filtrage de Kalman et l'affichage des coordonnées filtrées.

    Args:
        affichage (FunctionType): La fonction d'affichage à utiliser, choisie par l'utilisateur au lancement du programme. Elle doit prendre en argument un tuple de coordonnées (z,x,y).
    """
    arduino:serial.Serial = setup_arduino()
    filtres_inter, filtres_kalman = setup_filtres(arduino)
    try:
        while True:
            coords:list|None = lecture_donnees(arduino)
            # On vérifie que les coordonnées sont valides et dans les limites du bac avant de les ajouter à la mémoire et de les afficher
            if coords is not None and 0 < coords[1] < TAILLE_X_BAC and 0 < coords[2] < TAILLE_Y_BAC:
                coords_filtree:tuple = filtrage_kalman(filtres_inter,filtres_kalman,coords)
                affichage(coords_filtree)
```

```
            time.sleep(0.01)
    #Arrêt en faisant ctrl+c dans le terminal
    except KeyboardInterrupt:
        print(f"\nArrêt du programme...")
    #Arrêt en fermant la fenêtre d'affichage
    except RuntimeError as e:
        print(e)
    except serial.SerialException as e:
        print(f"Erreur : {e}")
    finally:
        # On s'assure de fermer la connexion série avec l'Arduino à la fin du programme
        if arduino.is_open:
            arduino.close()


if __name__ == "__main__":
    #Menu de sélection du mode d'affichage
    print("Choisissez un mode d'affichage :")
    for i, affichage in enumerate(MODULES_AFFICHAGES):
        print(f"{i+1}. {affichage[0]}")
    choix_valide:bool = False
    while not choix_valide:
        try:
            choix:int = int(input("Entrez le numéro du mode d'affichage : "))
            if 1 <= choix <= len(MODULES_AFFICHAGES):
                 # Import dynamique du module d'affichage choisi pour éviter de charger
tous les modules inutiles en mémoire
                  module_affichage:ModuleType =
                  import_module(f"affichages.{MODULES_AFFICHAGES[choix-1][1]}")
                choix_valide = True
            else:
                raise ValueError
        except ValueError:
            print(f"Veuillez entrer un numéro entre 1 et {len(MODULES_AFFICHAGES)}.")
        except ImportError as e:
            print(f"Module d'affichage mal implémenté : {e}")

    main(module_affichage.affichage)
```

lecture_serial.py :

```
"""Lecture des données d'une carte Arduino via une connexion série."""

import serial
import time
from collections import deque

from constante import *
from kalman import *

def setup_arduino()->serial.Serial:
    """Setup de la connexion série avec l'Arduino.

    Raises:
        TimeoutError: Si la configuration n'est pas reçue dans le délai imparti.

    Returns:
        serial.Serial: L'objet de connexion série configuré et prêt à être utilisé.
    """
    print("\nCréation de la connexion série...")
    arduino:serial.Serial = serial.Serial(PORT_ARDUINO, VITESSE, timeout=1)
    print(f"Connexion établie sur le port {PORT_ARDUINO}")
        timeout:float = time.time() + TIMEOUT_CONFIGURATION    # Timeout maximum de
configuration

    while time.time() < timeout:
        if arduino.in_waiting > 0:
            donnee:str = arduino.readline().decode('utf-8').rstrip()
            if donnee == "--- Setup OK ---":
                    print("Configuration arduino réussie, démarrage de la lecture des
données...")
                return arduino
    raise TimeoutError("Configuration non reçue dans le délai imparti.")

def setup_filtres(arduino:serial.Serial)->tuple:
     """On initialise les filtres pour éviter d'avoir des valeur fausses au démarrage du
programme

    Args:
        arduino (serial.Serial): L'objet de connexion série avec l'Arduino.
```

```
    Returns:
        tuple: Les filtres intermédiaire et de Kalman pour les 3 coordonnées
    """
       filtres_inter:tuple = (FiltreInter(TAILLE_MEMOIRE), FiltreInter(TAILLE_MEMOIRE), FiltreInter(TAILLE_MEMOIRE))
      filtres_kalman:tuple = (FiltreKalman(0.0, INCERTITUDE, Q, R), FiltreKalman(0.0, INCERTITUDE, Q, R), FiltreKalman(0.0, INCERTITUDE, Q, R))

    i = 0
     while i < TAILLE_MEMOIRE*3: # On lit au moins 3 fois la taille de la mémoire pour être sûr d'avoir des valeurs stables
        coords:list|None = lecture_donnees(arduino)
        if coords is not None:
            filtrage_kalman(filtres_inter,filtres_kalman,coords)
            i += 1
    print("Initialisation des filtres terminée.")
    return filtres_inter, filtres_kalman

def lecture_donnees(arduino:serial.Serial)->list|None:
    """Lecture des données écrite sur la connexion série.

    Args:
        arduino (serial.Serial): L'objet de connexion série avec l'Arduino.

    Returns:
         list|None: La liste des coordonnées lues [z,x,y]. On préfère renvoyer None que des listes par défaut pour éviter, en cas de lag de la carte arduino, de remplir la mémoire de valeurs fausses.
    """
    if arduino.in_waiting > 0:
        # Lecture de la ligne reçue
        donnee:str = arduino.readline().decode('utf-8').rstrip()
        return list(map(int, donnee.split(",")))
```

kalman.py : (cf Filtre de Kalman)

```
"""Module du filtre de Kalman"""

import collections
import numpy as np

from constante import *

class FiltreInter:
      """Filtre intermédiaire pour lisser les mesures avant de les passer au Kalman.
Utilise la médiane des valeurs.
    """
    def __init__(self, memoire:int):
        """Setup du filtre intermédiaire

        Args:
            memoire (int): Nombre de mesures à conserver pour le filtrage intermédiaire
        """
        self.valeurs:collections.deque = collections.deque(maxlen = memoire)


    def filtrer(self, nouvelle_valeur:float)->float:
        """Filtre la nouvelle valeur en utilisant la médiane des valeurs conservées

        Args:
            nouvelle_valeur (float): La nouvelle mesure à filtrer

        Returns:
            float: La valeur filtrée
        """
        self.valeurs.append(nouvelle_valeur)
        return float(np.mean(self.valeurs))

class FiltreKalman:
     """Filtre de Kalman pour estimer la position à partir des mesures filtrées par le
filtre intermédiaire.
    """
    def __init__(self, position:float, incertitude:float, q:float, r:float):
```

```
        """Setup du filtre de Kalman

        Args:
            position (float): La position initiale estimée
            incertitude (float): L'incertitude initiale
            q (float): Confiance de processus
            r (float): Confiance de mesure
        """
        self.position:float = position
        self.incertitude:float = incertitude
        self.q:float = q
        self.r:float = r


    def mettre_a_jour(self, mesure:float)->float:
        """Met à jour l'estimation avec la nouvelle mesure

        Args:
            mesure (float): La nouvelle mesure

        Returns:
            float: La valeur estimée
        """
        # 1. Étape de Prédiction (on suppose une vitesse constante à court terme)
        prediction:float = self.incertitude + self.q
        # 2. Étape de Correction (Calcul du gain de Kalman)
        k:float = prediction / (prediction + self.r)
        # Mise à jour de l'estimation avec la nouvelle mesure
        self.position = self.position + k * (mesure - self.position)
        # Mise à jour de l'incertitude
        self.incertitude = (1 - k) * prediction

        return self.position

def         filtrage_mesure(filtre_inter:FiltreInter,         filtre_kalman:FiltreKalman,
incertitude:float, q:float, r:float, mesure:int)->float:
    """filtre la mesure en utilisant le filtre de Kalman

    Args:
        filtre_inter (FiltreInter): Le filtre intermédiaire
        filtre_kalman (FiltreKalman): Le filtre de Kalman
```

```
        incertitude (float): L'incertitude initiale
        q (float): Confiance de processus
        r (float): Confiance de mesure
        mesure (int): La mesure à filtrer

    Returns:
        float: La valeur filtrée
    """

    valeur_brut:float = float(mesure)
    valeur_filtre_inter:float = filtre_inter.filtrer(valeur_brut)
    valeur_filtre_kalman:float = filtre_kalman.mettre_a_jour(valeur_filtre_inter)

    return valeur_filtre_kalman



def filtrage_kalman(filtres_inter:tuple,filtres_kalman:tuple,coords:list)->tuple:
    """Filtre les coordonnées en utilisant le filtre de Kalman

    Args:
        coords (list): La coordonnée brute à filtrer (z, x, y)
        filtres_inter (tuple): Les filtres intermédiaires pour les 3 coordonnées
        filtres_kalman (tuple): Les filtres de Kalman pour les 3 coordonnées

    Returns:
        tuple: Les coordonnées filtrées (z, x, y)
    """


            x_filtree:int   =   int(filtrage_mesure(filtres_inter[1],   filtres_kalman[1],
INCERTITUDE, Q, R, coords[1]))
            y_filtree:int   =   int(filtrage_mesure(filtres_inter[2],   filtres_kalman[2],
INCERTITUDE, Q, R, coords[2]))
            z_filtree:int   =   int(filtrage_mesure(filtres_inter[0],   filtres_kalman[0],
INCERTITUDE, Q, R, coords[0]))

    return (z_filtree, x_filtree, y_filtree)
```

affichages/affichage_console.py :

```
"""Module d'affichage des coordonnées brutes dans la console"""

def affichage(coords:tuple)->None:
    """Affichage des coordonnées brutes.

    Args:
        coords (tuple): coordonnées à afficher, sous la forme (z,x,y)
    """
    print(coords)

if __name__ == "affichages.affichage_console":
    #SETUP
    print("Affichage console sélectionné.")
```

affichages/affichage_console_raffine.py :

```
"""Module d'affichage des coordonnées brutes dans la console"""

def affichage(coords:tuple)->None:
    """Affichage des coordonnées brutes.

    Args:
        coords (tuple): coordonnées à afficher, sous la forme (z,x,y)
    """
    print(coords)

if __name__ == "affichages.affichage_console":
    #SETUP
    print("Affichage console sélectionné.")
```

affichages/affichage_graphique_instantane.py:

```
"""Module d'affichage graphique instantané des coordonnées mesurées par le LiDAR.
Il affiche un point sur un canvas en fonction des coordonnées fournies, avec une couleur qui dépend de la hauteur mesurée"""

from tkinter import Canvas
import tkinter as tk

def affichage(coords:tuple)->None:
    """Affiche un point sur un canvas en fonction des coordonnées fournies.

    Args:
        coords (tuple): Les coordonnées à afficher, sous la forme (z, x, y).

    Raises:
        RuntimeError: Si l'application graphique a été fermée.
    """
    try:
        #Calcule des coordonnées effectives du point à afficher sur le canvas
        x_effectif: int = coords[1]*COEFF_AGRANDISSEMENT
                                                                y_effectif:       int       =
TAILLE_Y_BAC*COEFF_AGRANDISSEMENT-coords[2]*COEFF_AGRANDISSEMENT  #L'axe  y  est  inversé
dans le canvas

             canvas.coords(point,  y_effectif  -  RAYON_POINT,  x_effectif  -  RAYON_POINT,
y_effectif + RAYON_POINT, x_effectif + RAYON_POINT)
        canvas.itemconfig(point, fill=couleur_depuis_hauteur(coords[0]))

        root.update()

    except Exception as e:
        if repr(e) == "TclError('invalid command name \".!canvas\"')":
                raise  RuntimeError  ("\nL'application  graphique  a  été  fermée.  Arrêt  du
programme...")
        print(f"Erreur lors de l'affichage : {e}")


if __name__ == "affichages.affichage_graphique_instantane":
    #SETUP
    from constante import *
```

```
    from affichages.couleur import couleur_depuis_hauteur

    print("Affichage graphique instantané sélectionné.")

    root:tk.Tk = tk.Tk()

     #Création du canvas et dessin de la grille, avec un agrandissement pour quelle soit plus grande à l'écran
                canvas:Canvas    =    Canvas(root,    width=TAILLE_Y_BAC*COEFF_AGRANDISSEMENT, height=TAILLE_X_BAC*COEFF_AGRANDISSEMENT, bg='white')
    canvas.pack()

    # Dessine une grille de carré de 5cm x 5cm pour l'échelle
            for   i   in   range(0,   max(TAILLE_Y_BAC,   TAILLE_X_BAC)*COEFF_AGRANDISSEMENT, 5*COEFF_AGRANDISSEMENT):
                                    canvas.create_line(i,    0,    i,    max(TAILLE_Y_BAC, TAILLE_X_BAC)*COEFF_AGRANDISSEMENT, fill='lightgray')
            canvas.create_line(0, i, max(TAILLE_Y_BAC, TAILLE_X_BAC)*COEFF_AGRANDISSEMENT, i, fill='lightgray')

    # Création du point qui représentera les coordonnées
                point    =    canvas.create_oval(0,    0,    RAYON_POINT*2,    RAYON_POINT*2, fill=COULEUR_HAUTEUR_MAX)
```

affichages/affichage_graphique_continue.py :

```
"""Module d'affichage graphique continu d'une grille de colorée en fonction des données du LiDAR."""

import tkinter as tk

from constante import *

class GrilleCouleur(tk.Tk):
    def __init__(self, lignes:int, colonnes:int, taille_case:int):
        """Setup de la fenêtre principale avec une grille de rectangles colorés.
            Chaque rectangle représente un carré d'un centimètre du bac, et sa couleur dépend de la dernière hauteur mesurée par le LiDAR.

        Args:
```

```
            lignes (int):Le nombre de lignes de la grille
            colonnes (int):Le nombre de colonnes de la grille
            taille_case (int):La taille de chaque case en pixels
        """
        super().__init__()
        self.title("Grille Tkinter - couleur par carreau")
        self.lignes:int = lignes
        self.colonnes:int = colonnes
        self.taille_case:int = taille_case
        self.couleur_initiale:str = couleur_depuis_hauteur(HAUTEUR_LIDAR)

        #Création du canvas et dessin de la grille
        self.canvas:tk.Canvas = tk.Canvas(
            self,
            width=colonnes * taille_case,
            height=lignes * taille_case,
            bg=self.couleur_initiale,
            highlightthickness=0,
        )
         #On ajoute du padding autour du canvas pour que les cases soient bien séparées du bords de la fenêtre
        self.canvas.pack(padx=10, pady=10)

        self.rectangles:dict = {}  #(ligne, colonne) -> clé associée à un rectangle

        self._dessiner_grille()

    def _dessiner_grille(self)->None:
         """Dessine la grille de rectangles sur le canvas, en stockant les IDs de chaque rectangle pour pouvoir les modifier plus tard."""
        for i in range(self.lignes):
            for j in range(self.colonnes):
                x1:int = j * self.taille_case
                y1:int = i * self.taille_case
                x2:int = x1 + self.taille_case
                y2:int = y1 + self.taille_case

                rect_id:int = self.canvas.create_rectangle(x1, y1, x2, y2, fill=self.couleur_initiale, outline="black")
                self.rectangles[(i, j)] = rect_id
```

```
    def changer_couleur_case(self, hauteur:int, ligne:int, colonne:int)->None:
        """Change la couleur d'une case de la grille en fonction de la hauteur mesurée.

        Args:
            hauteur (int):La hauteur mesurée par le LiDAR.
            ligne (int):La ligne de la case à modifier.
            colonne (int):La colonne de la case à modifier.
        """
        if not (0 <= ligne < self.lignes and 0 <= colonne < self.colonnes):
            return

        couleur:str = couleur_depuis_hauteur(hauteur)

        #On regroupe les cases par blocs de TAILLE_BLOCxTAILLE_BLOC
        ligne = (ligne // TAILLE_BLOC) * TAILLE_BLOC
        colonne = ((TAILLE_Y_BAC-colonne) // TAILLE_BLOC) * TAILLE_BLOC

        #On change la couleur de toutes les cases du bloc
        for dl in range(0, TAILLE_BLOC):
            for dc in range(0, TAILLE_BLOC):
                li = ligne + dl
                co = colonne + dc
                if 0 <= li < self.lignes and 0 <= co < self.colonnes:
                    rid = self.rectangles[(li, co)]
                    self.canvas.itemconfig(rid, fill=couleur)

        #rect_id = self.rectangles[(int(ligne), int(colonne))]
        #self.canvas.itemconfig(rect_id, fill=couleur)
         print(f"Case ({ligne}, {colonne}) mise à jour avec hauteur {hauteur} -> couleur
{couleur}")



def affichage(coords:tuple)->None:
    """Affiche un point sur une grille en fonction des coordonnées fournies.
        La grille est mise à jour en continu, avec une couleur qui dépend de la hauteur
mesurée.

    Args:
        coords (tuple):Les coordonnées à afficher, sous la forme (z, x, y).
```

```
    Raises:
        RuntimeError:Si l'application graphique a été fermée.
    """
    try:
        app.changer_couleur_case(*coords)
        app.update()
    except Exception as e:
        if repr(e) == "TclError('invalid command name \".!canvas\"')":
                raise RuntimeError ("\nL'application graphique a été fermée. Arrêt du
programme...")
        print(f"Erreur lors de l'affichage :{e}")


if __name__ == "affichages.affichage_graphique_continu":
    #SETUP
    from affichages.couleur import couleur_depuis_hauteur

    print("Affichage graphique continu sélectionné.")

                app    =    GrilleCouleur(lignes=TAILLE_X_BAC,    colonnes=TAILLE_Y_BAC,
taille_case=TAILLE_CASE)
```

constante.py :

```
"""Fichier contenant les constantes utiles au fonctionnement des programmes"""

#*Constante du monde réel
HAUTEUR_LIDAR  = 40 #En cm, la hauteur où est placé le LiDAR
HAUTEUR_EAU = 5 #En cm, la hauteur de l'eau dans le bac
TAILLE_X_BAC = 50 #En cm, la taille du bac en x, il vaut mieux une valeur plus grande
que la valeur réelle
TAILLE_Y_BAC = 30 #En cm, la taille du bac en y, idem


#*Constante Arduino
VITESSE = 115200 #En bauds, la vitesse de communication série avec l'Arduino. Doit être
la même que celle configurée dans le code de l'Arduino
PORT_ARDUINO = 'COM10' #Le port série où est connecté l'Arduino, peut se voir dans
l'Arduino IDE
TIMEOUT_CONFIGURATION = 10 #En secondes, le temps maximum pour recevoir la configuration
de l'Arduino après le démarrage du programme
```

```
#*Constante Filtrage de Kalman
TAILLE_MEMOIRE = 15
INCERTITUDE = 0.0 #Incertitude initiale du système
Q = 15.0 #Imprévisibilité de l'environnement
R = 3.0 #Erreur de mesure

#*Constante Choix du module d'affichage
#Les modules d'affichage doivent être dans le dossier affichages et implémenter une fonction affichage(coords:tuple)->None
#Couples (nom affiché dans le menu de choix, nom du module d'affichage) :
MODULES_AFFICHAGES = [("Affichage console ([z,x,y])", "affichage_console"),
                      ("Affichage console raffiné", "affichage_console_raffine"),
                                                    ("Affichage graphique instantané", "affichage_graphique_instantane"),
                      ("Affichage graphique continu", "affichage_graphique_continu")]


#*Constante Affichage Graphique Instantané
COEFF_AGRANDISSEMENT = 20 #Coefficient d'agrandissement de la fenêtre
RAYON_POINT = 15 #En pixels, le rayon du point représentant les coordonnées filtrées

#*Constante Affichage Graphique Continu
TAILLE_CASE = 10 #En pixels
TAILLE_BLOC = 2 #En nombre de cases, doit être un diviseur de NOMBRE_LIGNES et NOMBRE_COLONNES

#*Constante Couleur
RATIO_COULEUR_EAU = 0.9 #Le ratio de la couleur à la hauteur d'eau par rapport à la couleur au fond de l'eau
COULEUR_HAUTEUR_0 = "#00FF00" #Couleur pour la hauteur 0, c'est à dire quand un déchet 'touche' le capteur
COULEUR_HAUTEUR_MAX = "#0000FF" #Couleur pour la hauteur maximale de détection du lidar, c'est à dire quand il ne detecte rien
```

Code ESP 32 (cf Partie IV)

Connection WiFi entre l'ordinateur et l'ESP32

En C++: *Code provenant de la librairie WiFi dans Esp32-Arduino ; les annotations ont été traduites pour mieux comprendre le programme.*

→ libraries/WiFi/examples/SimpleWiFiServer/SimpleWiFiServer.ino

```
#include <WiFi.h>

const char *ssid = "WiFi_Esp32";#on connecte
const char *password = "LeMeIlLeUrMdP123";
int PinId = 2;
NetworkServer server(80);

void setup() {
  Serial.begin(115200);
     pinMode(PinId,  OUTPUT);     //  Définition  du  "pin  mode"  de  la  LED
(OUTPOUT=fournir du courant)

  delay(10);

  // On commence par se connecter au réseau WiFi

  Serial.println();
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());

  server.begin();
```

```
}

void loop() {
  NetworkClient client = server.accept();  // détecte les clients entrants

  if (client) {                     // si un client est connecté,
    Serial.println("New Client.");  // écrit un message dans le port série
     String currentLine = "";          // crée un texte pour garder les données entrantes provenant du client
    while (client.connected()) {    // boucle while le client est connecté
       if (client.available()) {      // s'il y a des octets à lire provenant du client,
        char c = client.read();     // lire l'octet puis,
        Serial.write(c);            // l'écrire dans le moniteur série
        if (c == '\n') {            // si l'octet est un "new line character"

           // si l'actuelle ligne est blank, alors on obtient 2 "newlines" à la suite.
          // Fin de la requête client HTTP, donc envoyer la réponse:
          if (currentLine.length() == 0) {
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type:text/html");
            client.println();

            // le contenu de la réponse HTTP suit l'en tête:
             client.print("Click <a href=\"/H\">here</a> to turn the LED on pin 2 on.<br>");
             client.print("Click <a href=\"/L\">here</a> to turn the LED on pin 2 off.<br>");

            // La réponse HTTP finit par une autre ligne blank:
            client.println();
            // break de la boucle:
            break;
          } else {  // s'il y a une newline, alors effacer la currentLine:
            currentLine = "";
          }
         } else if (c != '\r') {  // si une autre réponse est obtenue qui n'est pas un paquet de données retourner un caractère,
          currentLine += c;      // l'ajouter à la fin de la currentLine
        }
```

```
          // Vérifie si la requête du client est un "GET /H"(allumer la LED) ou un
"GET /L"(éteindre la LED):
        if (currentLine.endsWith("GET /H")) {
          digitalWrite(PinId, HIGH);  // GET /H allume la LED        }
        if (currentLine.endsWith("GET /L")) {
          digitalWrite(PinId, LOW);  // GET /L éteins la LED
        }
      }
    }
    // fermer la connection avec le client:
    client.stop();
    Serial.println("Client Disconnected.");
  }
}
```